



**UNIVERSITATEA
TEHNICĂ
DIN CLUJ-NAPOCA**

**Detectarea accelerata de FPGA a componentelor
conexe in imagini binare**

Structura Sistemelor de Calcul

Asistent de laborator: Corpodean Darius

Student: Stoica Mihai-Nicutor

Grupa: 30237

FACULTATEA DE AUTOMATICA
SI CALCULATOARE

26 Noiembrie 2025

Cuprins

Rezumat	2
1 Introducere	3
1.1 Contextul si definirea problemei	3
1.2 Obiectivele proiectului	3
1.3 Solutia propusa	3
1.4 Structura raportului	3
2 Fundamentare teoretica	3
2.1 Algoritmul Connected Component Labeling (CCL)	3
2.2 Platforma PYNQ si interfete AXI	4
3 Proiectare si implementare	4
3.1 Arhitectura generala a sistemului	4
3.2 Descrierea modulelor hardware	6
3.2.1 Modulul de feliere (ccl_slice_core)	6
3.2.2 Modulul de re-etichetare (ccl_relabel_core)	6
3.3 Ghid de utilizare si Control Software	7
3.4 Configuratia hardware reala	8
4 Rezultate experimentale	8
4.1 Metodologia de testare	8
4.2 Analiza vizuala a procesarii intermediare	8
4.3 Analiza rezultatelor si Comparatie de Performanta	10
5 Concluzii	10
5.1 Sinteza realizarilor	10
5.2 Dezvoltari viitoare	10
A Cod sursa si scheme	12

Rezumat

Acest proiect prezinta proiectarea si implementarea unui sistem hardware de etichetare a componentelor conexe (Connected Component Labeling - CCL) accelerat pe platforma PYNQ-Z1. Obiectivul principal este procesarea eficienta a imaginilor binare cu rezolutia de 512x512 pixeli prin paralelizarea operatiilor. Solutia propusa utilizeaza o arhitectura Zynq-7000, unde partea de procesare intensiva este transferata in logica programabila (PL). Imaginea este divizata in patru segmente orizontale de 512x128 procesate simultan de nuclee hardware dedicate, care comunica printr-o memorie partajata pentru gestionarea echivalentelor dintre etichete. Rezultatele sunt unificate de un nucleu final de re-etichetare, demonstrand fezabilitatea si performanta arhitecturilor paralele in aplicatiile de viziune computerizata.

1 Introducere

Procesarea imaginilor in timp real necesita resurse computationale semnificative, in special pentru algoritmi care implica parcurgerea si analiza fiecarui pixel, cum este etichetarea componentelor conexe (CCL). Aceasta sectiune plaseaza tema proiectului in contextul accelerarii hardware folosind FPGA-uri.

1.1 Contextul si definirea problemei

Algoritmii clasici de CCL rulati pe procesoare de uz general (CPU) executa operatiile secvential, ceea ce limiteaza performanta la rezolutii mari. In contextul sistemelor embedded, cum ar fi PYNQ-Z1, utilizarea eficienta a logicii programabile (PL) poate oferi un grad ridicat de paralelism. Problema abordata este implementarea unui algoritm Two-Pass CCL care sa imparta sarcina de procesare intre mai multe unitati de executie hardware pentru a reduce latenta totala [2].

1.2 Obiectivele proiectului

Obiectivele principale sunt:

- Implementarea algoritmului CCL pe FPGA utilizand limbajul VHDL.
- Paralelizarea procesului prin divizarea imaginii in slice-uri (felii) procesate independent.
- Realizarea comunicatiei eficiente intre procesor (PS) si logica programabila (PL) prin DMA si interfete AXI.
- Gestionarea conflictelor de etichete utilizand o memorie BRAM partajata.

1.3 Solutia propusa

Solutia implica un design hardware modular. Imaginile sunt pre-procesate in Python (OpenCV) si trimise in memoria DDR. FPGA-ul preia aceste imagini prin DMA-uri, le imparte in 4 sectiuni egale de 512x128 pixeli si le proceseaza in paralel pentru a genera etichete provizorii. Un modul final parcurge imaginea reconstruita pentru a rezolva echivalentele si a genera etichetele finale, care sunt afisate ulterior prin interfata HDMI.

1.4 Structura raportului

Sectiunea 2 detaliaza fundamentele teoretice ale algoritmului CCL. Sectiunea 3 descrie arhitectura hardware si fluxul de date. Sectiunea 4 prezinta rezultatele obtinute, iar Sectiunea 5 contine concluziile si directiile viitoare.

2 Fundamentare teoretica

Aceasta sectiune descrie algoritmul de baza utilizat si tehnologiile specifice platformei Xilinx Zynq.

2.1 Algoritmul Connected Component Labeling (CCL)

CCL este un algoritm fundamental in prelucrarea imaginilor folosit pentru a detecta regiuni conexe in imagini binare. Metoda specifica utilizata in aceasta lucrare este preluata si adaptata

din resursele laboratorului de Prelucrarea Imaginilor al domnului profesor Giosan [4], fiind un algoritm de tip "Two-Pass" optimizat pentru structuri regulate de memorie.

Principiile generale ale algoritmului sunt discutate si in literatura de specialitate si tutoriale video dedicate accelerarii pe FPGA [3] [6]. Varianta aleasa a fost preferata deoarece reprezinta cel mai eficient mod de a paraleliza procesarea prin impartirea imaginii in felii (slices) independente in prima faza, desi necesita o a doua trecere secventiala pentru reuniunea finala.

Algoritmul presupune urmatoorii pasi:

1. **Prima trecere (Paralelizabila):** Se parcurge imaginea pixel cu pixel. Se atribuie etichete temporare pe baza vecinilor (stanga si sus). Daca exista un conflict (vecinii au etichete diferite, dar fac parte din acelasi obiect), se noteaza echivalenta intr-un tabel (LUT - Look-Up Table). Aceasta etapa se executa simultan pe 4 nuclee hardware in proiectul nostru.
2. **A doua trecere (Reuniune):** Se parcurge imaginea etichetata temporar si se inlocuieste fiecare eticheta cu "radacina" sa din tabelul de echivalente, unificand astfel regiunile conectate.

2.2 Platforma PYNQ si interfete AXI

Proiectul este implementat pe o placa de dezvoltare PYNQ-Z1 [1]. Aceasta integreaza un sistem de procesare (PS) ARM Cortex-A9 si logica programabila (PL) Artix-7. Comunicatia dintre module se realizeaza prin protocolul AXI:

- **AXI Stream:** Pentru transferul rapid al fluxului de pixeli (imagini).
- **AXI Lite:** Pentru configurarea registrelor de control si status ale modulelor proprii.
- **AXI Full/Master:** Pentru accesul la memoria partajata (BRAM) unde este stocat tabelul de echivalente.

3 Proiectare si implementare

Aceasta este partea principala a proiectului si descrie arhitectura sistemului implementat in Vivado si logica modulelor VHDL.

3.1 Arhitectura generala a sistemului

Sistemul este compus dintr-o parte software (Python pe Linux) si o parte hardware (PL). Procesarea nu se realizeaza in timp real pe un flux video continuu, ci pe imagini statice incarcate in memorie, permitand analiza detaliata a performantei pentru fiecare cadru.

In Figura 1 este prezentata diagrama bloc a sistemului, evidentiind interconectarea dintre nucleele de procesare si memoria sistemului.

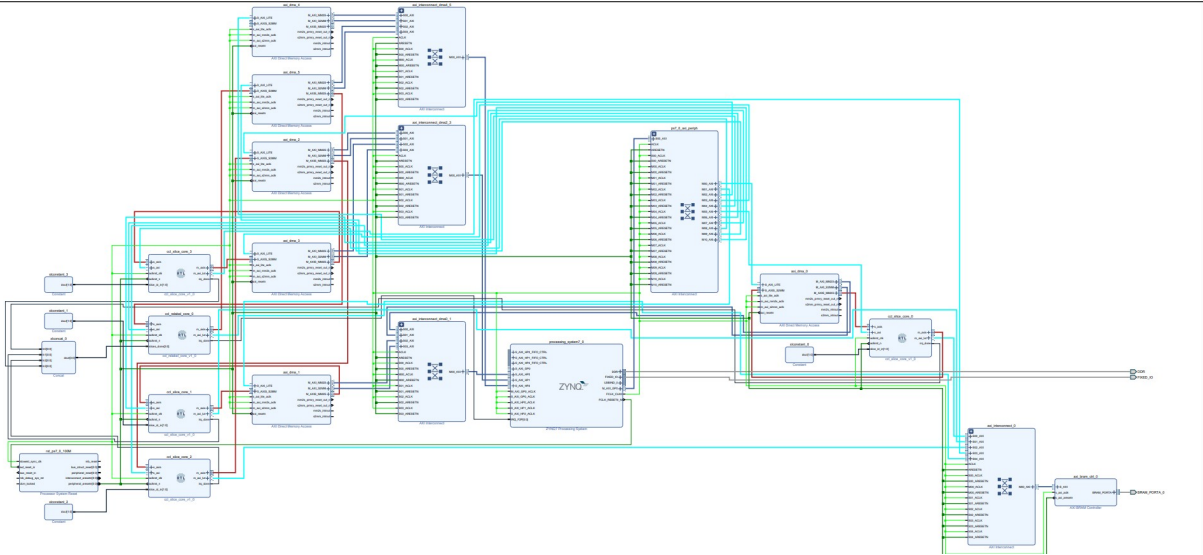


Figura 1: Schema bloc a sistemului in Vivado

Figurile urmatoare prezinta detaliile de implementare ale nucleelor IP create pentru acest proiect. Figura 2 ilustreaza interfețele modului de feliere, iar Figura 3 prezinta modulul de re-etichetare.

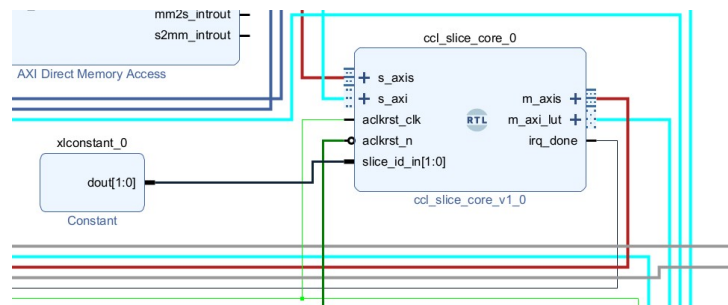


Figura 2: Interfața nucleului CCL Slice Core (Pass 1)

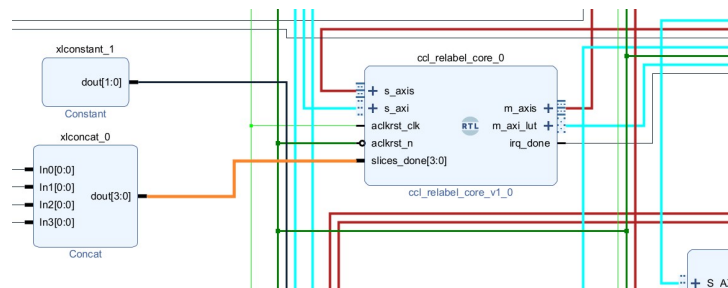


Figura 3: Interfața nucleului CCL Relabel Core (Pass 2)

Componentele hardware utilizate in acest proiect sunt:

- **Placa de dezvoltare PYNQ-Z1:** Echipata cu SoC Xilinx Zynq-7000 (procesor ARM Cortex-A9 + FPGA Artix-7).
- **Adaptor USB-Ethernet:** Utilizat pentru conectivitatea la retea si controlul placii prin Jupyter Notebook din browser.
- **Card MicroSD:** Contine imaginea sistemului de operare PYNQ Linux (bazat pe Ubuntu).

- **Monitor/Ecran LCD:** Conectat prin portul HDMI out al placii pentru vizualizarea rezultatului final (imaginea etichetata color).
- **Cabluri:** Cablu HDMI pentru iesirea video si cabluri USB pentru alimentare.

Schema bloc interna din Vivado cuprinde:

- **4 x ccl_slice_core:** Module identice care proceseaza paralel cate o felie din imagine.
- **1 x ccl_relabel_core:** Modulul care finalizeaza etichetarea.
- **6 x AXI DMA:** Pentru transferul datelor intre memoria DDR si nucleele de procesare si intre PS si PL.
- **AXI BRAM Controller:** Oferă acces partajat la memoria BRAM interna pentru gestionarea tabelului de echivalente (LUT).
- **Interconectari AXI:** Gestioneaza arbitrarea accesului la memorie.

Pentru a optimiza transferul de date si a evita gaturile (bottlenecks) la accesul in memoria DDR, unitatile DMA au fost distribuite strategic pe porturile de inalta performanta ale procesorului Zynq. Designul utilizeaza trei module AXI Interconnect conectate la porturile HP0, HP1 si HP2. Unitatile DMA sunt grupate cate doua pe un interconnect (si implicit pe un port HP), asigurand o latime de banda suficienta pentru scrierea si citirea paralela a imaginilor.

3.2 Descrierea modulelor hardware

3.2.1 Modulul de feliere (ccl_slice_core)

Acest modul implementeaza prima trecere a algoritmului CCL. Primeste un stream de 32 de biti (impachetati) reprezentand pixeli binari.

- **Intrare:** O portiune de 512x128 pixeli din imaginea totala.
- **Functionare:** Modulul despacheteaza bitii si analizeaza conectivitatea 8-way sau 4-way. Mentine un contor local pentru etichete. Cand intalneste un conflict intre eticheta vecinului din stanga si cel de sus, scrie o pereche de echivalenta in LUT-ul global din BRAM prin interfata AXI Master.
- **Iesire:** Un stream de etichete preliminare (32-bit) trimis inapoi in memorie.

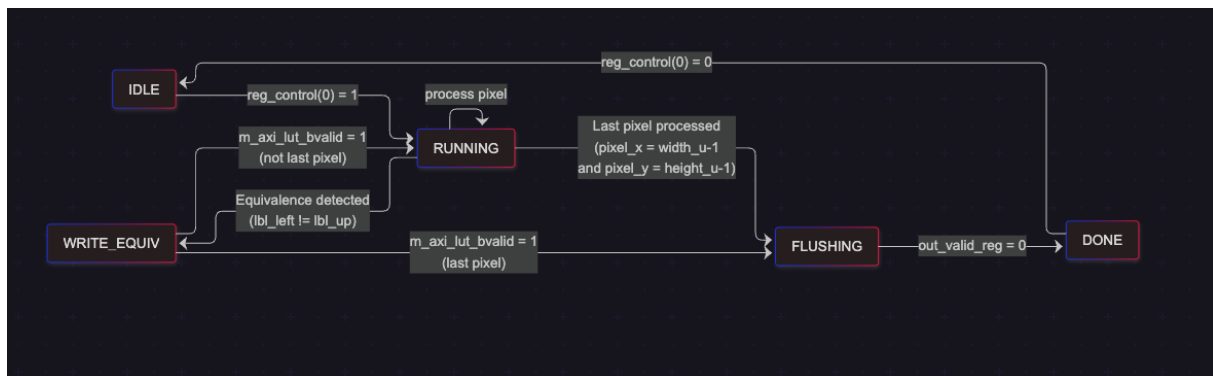


Figura 4: Diagrama de stari (FSM) pentru ccl slice core

3.2.2 Modulul de re-etichetare (ccl_relabel_core)

Acest modul este activat dupa ce toate cele 4 slice-uri au terminat procesarea. Semnalele de "Done" sunt concatenate pentru a semnaliza startul.

- **Curatare LUT:** Modulul are capacitatea de a initializa memoria BRAM cu zerouri inainte de procesare.

- **Rezolvarea echivalentelor:** Citeste stream-ul de etichete preliminare (imaginea intreaga recombina) si pentru fiecare pixel interogheaza LUT-ul din BRAM pentru a gasi eticheta radacina (canonica).
- **Iesire:** Imaginea finala etichetata corect.
- din cauza blocarii resurselor pe DMA-uri, FPGA-ul are probleme in a functiona dupa completarea slice-urilor, deci pentru testare acest pas de relabel este implementat software in notebook.

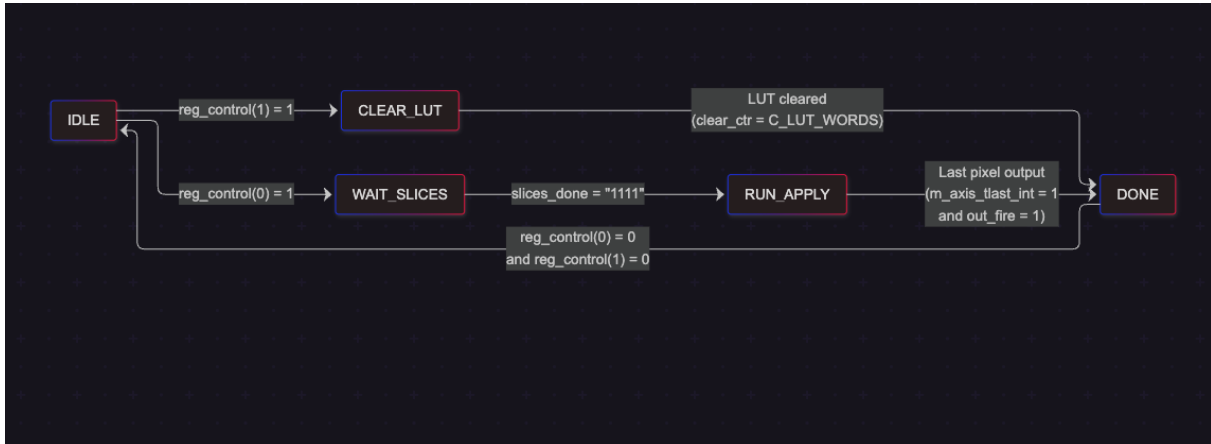


Figura 5: Diagrama de stari (FSM) principala pentru nucleul de re-etichetare

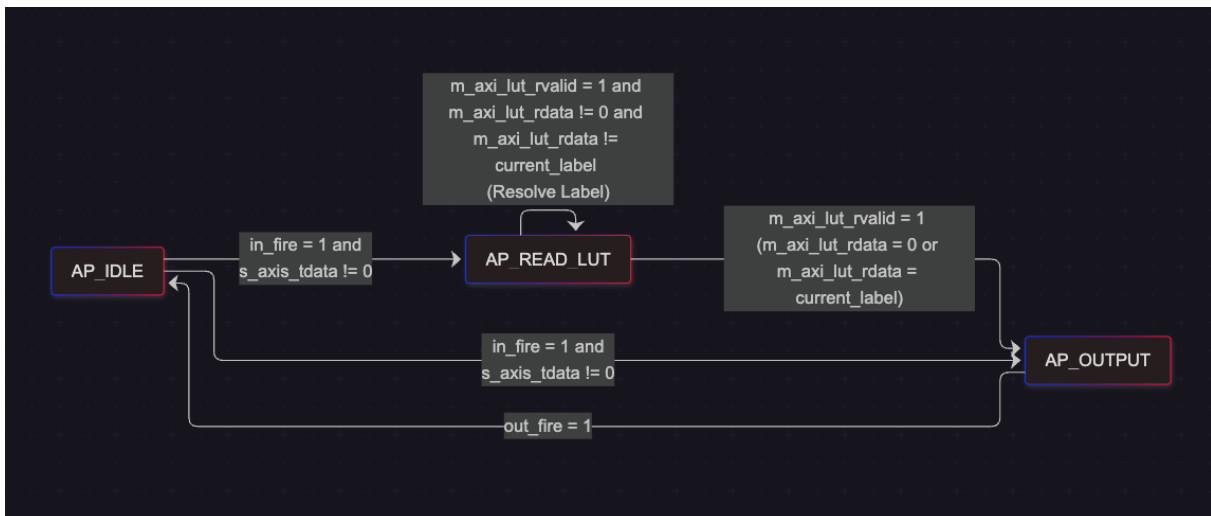


Figura 6: Sub-FSM-ul "Apply" utilizat pentru aplicarea etichetelor finale

3.3 Ghid de utilizare si Control Software

Pentru a utiliza sistemul, este necesara parcurgerea urmatoarelor pasi:

1. Se alimenteaza placa PYNQ-Z1 si se conecteaza la aceeasi retea cu PC-ul gazda prin adaptorul Ethernet.
2. Se acceseaza serverul Jupyter ruland in browser adresa IP a placii.
3. Din interfata web Jupyter, se navigheaza catre folderul **proiect** si se deschide notebook-ul principal (**main.ipynb**).
4. Se ruleaza celulele notebook-ului secvential pentru a incarca bitstream-ul (**.bit**) si a incepe procesarea.

Fluxul controlat din Python include:

1. Pre-procesarea imaginii in Python, inclusiv binarizarea folosind functiile de thresholding din biblioteca OpenCV [5] (redimensionare la 512x512 si impachetare biti).
2. Initializarea DMA-urilor si a registrelor de control pentru nucleele hardware.
3. Lansarea executiei paralele: cele 4 nuclee slice pornesc simultan.
4. Sincronizarea si lansarea nucleului de relabeling.
5. Citirea rezultatului final si vizualizarea imaginii etichetate pe ecranul conectat la HDMI.
6. Pentru inca o rulare este necesara repornirea placii pentru a debloca resursele FPGA-ului.

3.4 Configuratia hardware reala

In Figura 7 este prezentata implementarea fizica a sistemului descris anterior.



Figura 7: Implementarea fizica a sistemului: Placa PYNQ-Z1 conectata la ecranul LCD prin HDMI

4 Rezultate experimentale

In aceasta sectiune sunt prezentate validarea functionalitatii si o analiza comparativa a performantei fata de o solutie software.

4.1 Metodologia de testare

Sistemul a fost testat utilizand imagini standard de test. Imaginile de intrare sunt convertite in format binar si transferate in memoria DDR a placii. Pentru comparatie, s-a utilizat o varianta software rulata pe procesorul ARM (JustPythonBenchmark.ipynb).

4.2 Analiza vizuala a procesarii intermediare

Un aspect cheie al proiectului este paralelizarea prin divizarea imaginii in slice-uri. In Figura 12 este prezentata evolutia vizuala a procesului de etichetare folosind imaginile rezultate din sistem.

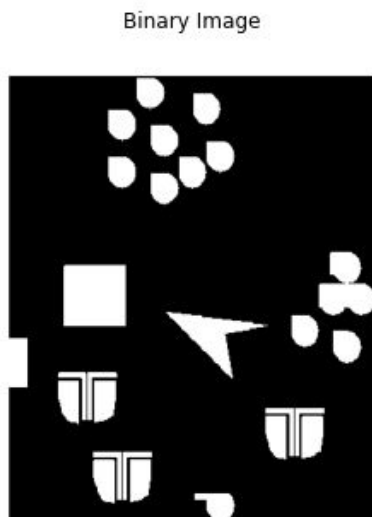


Figura 8: (a) Imagine binara de intrare



Figura 9: (b) Iesirea celor 4 nuclee (Pass 1)



Figura 10: (c) Imagine reconstruita (cu conflicte)

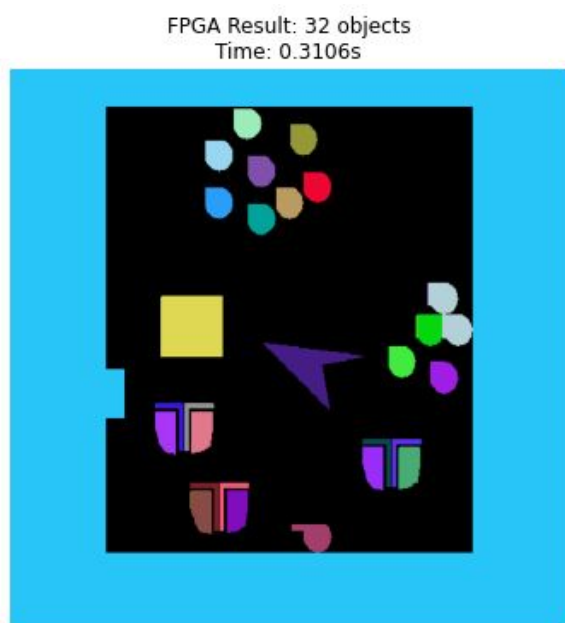


Figura 11: (d) Rezultat final (Relabeled)

Figura 12: Vizualizarea etapelor de procesare hardware: De la imaginea binara pana la unificarea finala a etichetelor.

Imaginea (b) arata clar cum fiecare nucleu hardware lucreaza pe o banda orizontala, atribuind etichete incepand de la 1 in cadrul feliilor lor. Imaginea (c) ilustreaza problema de rezolvat: obiectele care trec prin granita feliilor au culori diferite. In final, imaginea (d) demonstreaza succesul pasului de **relabeling**, care unifica aceste regiuni intr-un singur obiect conex intr-un timp de aproximativ 0.3 secunde.

4.3 Analiza rezultatelor si Comparatie de Performanta

Tabelul de mai jos compara timpii de executie obtinuti:

Implementare	Platforma	Timp Executie slices	Timp Relabel
Software Python	ARM Cortex-A9 (PS)	~ 2.5 secunde	~ 0.5 secunde
Hardware FPGA	Artix-7 FPGA (PL)	~ 0.0118 secunde	~ 0.307 secunde

Tabela 1: Comparatie performanta Software vs Hardware

Se observa ca implementarea hardware este cu ordine de marime mai rapida. Paralelizarea pe 4 nuclee si pipeline-ul hardware elimina bottleneck-urile secventiale specifice executiei pe CPU.

Dificultatile intalnite au inclus gestionarea corecta a ordinii bitilor (Big Endian vs Little Endian) la impachetarea pixelilor si sincronizarea accesului la BRAM prin interfata AXI.

5 Concluzii

Acest proiect a demonstrat implementarea cu succes a unui accelerator hardware pentru etichetarea componentelor conexe pe o arhitectura eterogena, avand o accelerare a timpului de executie al slice-urilor de aprox. 211 ori.

5.1 Sinteza realizarii

S-a realizat un design complex care integreaza nuclee IP proprii (VHDL), DMA-uri si interconectari AXI. Abordarea paralela (slicing) a permis procesarea simultana a datelor, iar utilizarea BRAM-ului ca memorie partajata a rezolvat problema unificarii etichetelor intre regiuni diferite.

5.2 Dezvoltari viitoare

Pentru viitor, se propun urmatoarele imbunatatiri:

- Deblocarea resurselor FPGA dupa pasul slice-urilor pentru a putea testa proiectul in intregime, incluzand rularea relabel-ului, fara a reporni placa.
- Optimizarea algoritmului de scriere in LUT pentru a reduce congestia pe magistrala AXI.
- Implementarea unui mecanism de "Union-Find" complet in hardware.
- Extinderea suportului pentru rezolutii video HD (1920x1080) prin cresterea numarului de slice-uri.

Bibliografie

- [1] Digilent, *PYNQ-Z1 Reference Manual*, Accesat: 2024, 2024.
- [2] DLR, „Parallel Connected Component Labeling for FPGA-Based Image Processing Systems”, în *DLR Electronic Library* (2015), Accesat: 2024.
- [3] FPGAVision, *Tutorial utilizare AXI DMA si PYNQ Overlay*, <https://www.youtube.com/watch?v=2ErFDGSv5EE>, Video YouTube.
- [4] Ion Giosan, *Resurse Laborator Prelucrarea Imaginilor - Lucrarea 5*, <https://users.utcluj.ro/~igiosan/Resources/PI/L5/PI-L5r.pdf>, Accesat: 2024, 2024.
- [5] OpenCV, *Image Thresholding - OpenCV 4.x Documentation*, Accesat: 2024, 2024.
- [6] Udacity, *Tutorial AXI DMA si PYNQ Overlay (Suport Curs)*, https://www.youtube.com/watch?v=Ld01yPmW_Xw, Video YouTube.

A Cod sursa si scheme